课程笔记

CS336 第 5 讲：GPU、内存墙与高性能矩阵乘法直觉

Codex 基于 Stanford CS336 2025 第 5 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 05 月 01 日
视频时长：1:14:21
视频链接：https://www.youtube.com/watch?v=6OBtO9niT00

目录

1 为什么语言模型时代必须真正理解 GPU **2**

1.1 为什么 GPU 话题和 CS336 主线天然一致 2

1.2 本章小结 2

2 GPU 与 CPU 的差别：吞吐机器而不是低延迟机器 **3**

2.1 CPU 优化的是少量快速线程，GPU 优化的是海量并行线程 3

2.2 为什么语言模型如此适合 GPU 3

2.3 SM、thread、block、warp：为什么 GPU 的并行不是“无限同时” 3

2.4 本章小结 4

3 GPU 为什么会慢：memory hierarchy 与 memory wall **4**

3.1 越靠近计算单元的存储越快，也越贵 4

3.2 真正限制大模型系统速度的，常常不是算不动，而是喂不饱 4

3.3 TPU 为什么在高层上和 GPU 很像 5

3.4 本章小结 5

4 怎样让 GPU 真正跑快：六种常见思路 **5**

4.1 控制分支发散：SIMT 的第一条纪律 5

4.2 低精度计算：先从少搬一点数据开始 6

4.3 operator fusion：别把中间结果来回运来运去 6

4.4 recomputation：有时多算一点反而更快 6

4.5 memory coalescing：为什么连续对齐访问那么重要 6

4.6 tiling：这讲真正的大招 6

4.7 本章小结 7

5 从矩阵乘法到 FlashAttention：GPU 直觉如何落到真实 LM kernel **7**

5.1 为什么这讲最后要回到矩阵乘法之谜 7

5.2 FlashAttention 的意义：让 attention 更像硬件愿意接受的矩阵程序 7

5.3 本章小结 7

6 总结与延伸 **7**

1 为什么语言模型时代必须真正理解 GPU

第 5 讲的起点不是“怎么写 CUDA kernel”，而是一个更宏观的判断：**今天大模型的进步，已经不能只从算法论文里理解，必须同时从硬件扩展、并行性和利用率的角度去理解**。如果第 2 讲让我们学会了用 FLOPs 和显存去记账，那么这一讲就是在问：这些 FLOPs 最终到底是在哪种机器上被执行的？又为什么同样的矩阵乘法，在 GPU 上会快得离谱，而在一些情况下又会突然慢下来？

课程开头就给出一个非常重要的 framing：很多时候，compute 的增长会直接带来语言模型性能的可预测提升。换句话说，**硬件变快、利用率提高、并行扩展改善，本身就足以推动一代代语言模型继续向前**。

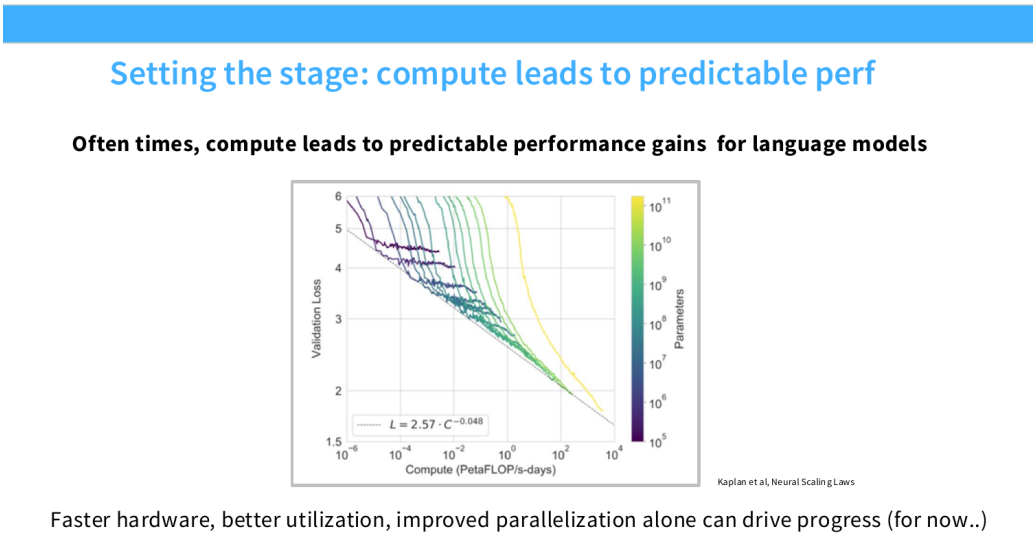

图 1: 第 5 讲开场就把 scaling law 视角和 GPU 讨论绑在一起：算力增长不是背景音，而是语言模型进展的主驱动力之一。

1.1 为什么 GPU 话题和 CS336 主线天然一致

这门课从第 1 讲开始就不断强调“资源约束下做联合优化”。GPU 恰好是这个命题最具体的载体之一。你是否能把矩阵乘法喂饱，是否能让访存对齐，是否能减少不必要的数据搬运，最后都会直接体现为 wall-clock time、训练预算和推理吞吐。

第 5 讲的核心任务

让 GPU 不再像魔法黑盒，而是让你明白：为什么它快、为什么它会慢、以及怎样设计一个更符合 GPU 偏好的算法。

1.2 本章小结

这一讲承接第 2 讲的资源会计，把“计算”从抽象 FLOPs 变成真实硬件对象。只有把 GPU 看作具体执行机器，你才会真正理解后面关于 kernel、tiling、FlashAttention 和并行训练的所有内容。

2 GPU 与 CPU 的差别：吞吐机器而不是低延迟机器

2.1 CPU 优化的是少量快速线程，GPU 优化的是海量并行线程

课程对 GPU 与 CPU 的对比非常简洁，但特别有用。CPU 的设计重点是低延迟，它擅长把少数线程尽快执行完，因此会为控制、缓存和分支投入更多硬件资源；GPU 的设计重点则是高吞吐，它假设你有大量结构相似的计算任务，于是把更多面积让给海量算术单元。

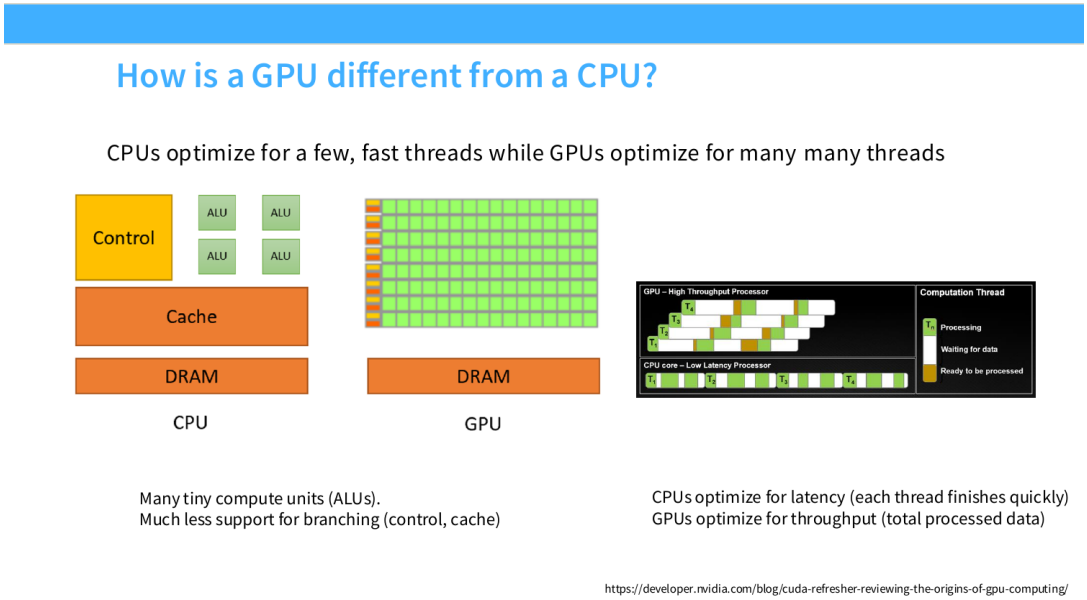

图 2: CPU 与 GPU 的根本差别在于优化目标：前者更偏低延迟控制流，后者更偏高吞吐并行计算。

这意味着，GPU 不是“更强的 CPU”，而是一台完全不同哲学的机器。你不能拿依赖复杂控制流和频繁分支的算法，指望它在 GPU 上天然跑得快；相反，GPU 喜欢的是大量相似操作同时发生，最好还是矩阵乘法这种结构规整的工作负载。

2.2 为什么语言模型如此适合 GPU

语言模型训练与推理高度依赖大规模线性代数，尤其是矩阵乘法。这种工作负载非常适合 GPU：同一类操作要在很多 token、很多 batch、很多 head 上重复发生，天然适合并行展开。课程还回顾了一个历史点：早期研究者甚至会想办法把图形硬件“hack”成矩阵乘法器，而后来的 tensor cores 则直接把这种需求变成了硬件一等公民。

一个很重要的心智模型

如果你的问题可以被整理成“大量规则、重复、矩阵化的计算”，GPU 往往会非常喜欢你；如果你的问题充满细碎分支、随机访存和不规则依赖，GPU 就会开始显得别扭。

2.3 SM、thread、block、warp：为什么 GPU 的并行不是“无限同时”

课程随后讲执行模型。最关键的三个层次是：

- **thread**：最细粒度执行单元，真正“干活”的线程。
- **block**：一组 thread，通常共享同一块 shared memory。

- **warp**：32 个连续线程一起执行同一条指令，是 SIMT 模式的基本执行束。

这件事非常重要，因为很多性能问题都来自你忽略了 warp 这个层次。例如一旦线程在同一个 warp 内走了不同分支，就会出现 control divergence，吞吐会立刻下降。

2.4 本章小结

GPU 与 CPU 的差别并不是“一个快一个慢”这么简单，而是“一个优化延迟，一个优化吞吐”。语言模型之所以依赖 GPU，不是因为 GPU 在任何事上都更强，而是因为它特别擅长大规模并行线性代数。

3 GPU 为什么会慢：memory hierarchy 与 memory wall

3.1 越靠近计算单元的存储越快，也越贵

课程在 memory hierarchy 这一块给出了一个贯穿后面所有技巧的总原则：**离 SM 越近的存储越快，但容量更小、成本更高**。 shared memory / L1 cache 很快，L2 次之，HBM / global memory 更远更慢，但容量大。

这和 CPU 时代的缓存直觉有点像，但在 GPU 上它变得更极端，因为吞吐太高、线程太多，任何一次“要去远处拿数据”的动作都会放大成大问题。

3.2 真正限制大模型系统速度的，常常不是算不动，而是喂不饱

第 5 讲最值得记住的图之一，就是“compute scaling is faster than memory scaling”。过去十几年里，硬件峰值 FLOPs 的增长远快于 DRAM 带宽和互连带宽的增长，于是系统越来越容易进入一种典型局面：**算力已经很强，但数据送不过来**。

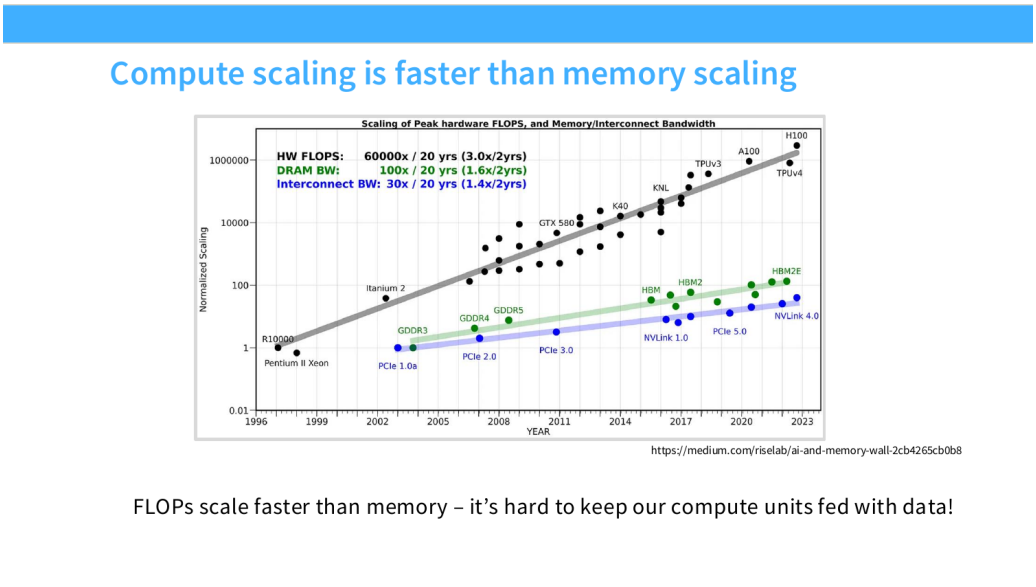

图 3: compute 增长快于 memory / interconnect 增长，这就是现代 GPU 上经典的 memory wall。

这张图几乎可以当作后面一整串优化技巧的总注脚。为什么大家要做低精度？因为要少搬点数据。为什么要 operator fusion？因为别来回写回 global memory。为什么要 tiling？因为想把重复访问

的数据尽量留在更近的存储层。为什么 FlashAttention 重要？因为 attention 的瓶颈不仅是 FLOPs，更是访存模式。

关于“高 FLOPs 就一定快”的误区

很多大模型实现的问题，不在于理论 FLOPs 不够高，而在于算力和数据搬运之间严重失衡。只盯 FLOPs 而不盯 bandwidth，会让你对真实性能做出系统性误判。

3.3 TPU 为什么在高层上和 GPU 很像

课程在一页 side thread 里顺便谈了 TPU。这个比较的重点不是品牌对比，而是帮助读者抽象出一层更高的 accelerator 视角：不管叫 GPU、TPU 还是别的 accelerator，核心都还是“轻量控制 + 快速矩阵单元 + 高速近端存储 + 更大的远端内存”。区别更多体现在网络互连方式、执行模型细节以及非 matmul 工作负载的权衡上。

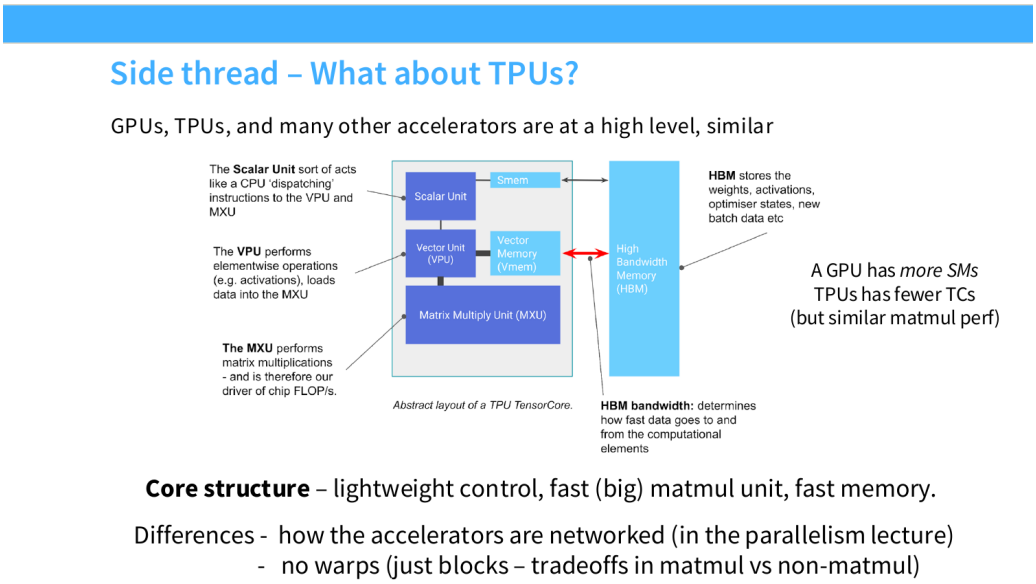

图 4: TPU 对照页的作用不是偏题，而是提醒你：accelerator 设计的高层结构其实是相似的。

3.4 本章小结

理解 GPU 慢在哪里，比理解 GPU 快在哪里更重要。因为大模型时代绝大多数性能工程，都是在和 memory wall 纠缠。你不先接受“compute 比 memory 扩得快得多”这个现实，就很难真正看懂后面的优化策略。

4 怎样让 GPU 真正跑快：六种常见思路

4.1 控制分支发散：SIMT 的第一条纪律

课程列出的第一条技巧是 control divergence。因为同一 warp 内线程倾向一起执行同样指令，所以一旦分支条件不一致，就会出现“这部分线程先等着、那部分线程先跑”的低效情况。这个问题不是 memory bottleneck，但在 GPU 上照样致命。

4.2 低精度计算：先从少搬一点数据开始

低精度的价值，在第 2 讲已经从训练角度解释过；到了第 5 讲，它的意义又多了一层：**更少比特意味着更少数据搬运，往往也意味着更高算术强度**。如果一个 ReLU 操作在 float32 下需要搬运 8 bytes，而在 float16 下只需要 4 bytes，那么在其他条件近似不变时，低精度就会直接提升“每搬运一个字节能做多少有用工作”。

4.3 operator fusion：别把中间结果来回运来运去

课程用非常直观的工厂/仓库比喻解释 fusion。global memory 像远处仓库，计算单元像工厂；如果每个小操作都要把中间结果写回仓库，再读出来交给下一个操作，那就相当于不停卡在物流上。fusion 的本质，是**把一串本来会反复读写 global memory 的操作合并进同一个 kernel，尽量在局部完成**。

4.4 recomputation：有时多算一点反而更快

recomputation 乍看违反直觉，因为它故意丢掉中间激活，再在需要时重新算。但课程正是想说明，GPU 上“再算一次”和“把一大堆激活从远处搬回来”之间，并不总是前者更贵。在 memory wall 很严重时，适度重算反而可能是更优策略。

4.5 memory coalescing：为什么连续对齐访问那么重要

GPU 的 global memory 读取常常是 burst mode。如果同一 warp 的线程访问地址分散、不对齐，那么一次访问就会浪费大量带宽；如果访问是对齐、连续的，就能更好地 coalesce 到同一 burst 中。矩阵布局、线程映射方式和 tile 形状都会直接影响这里的效率。

4.6 tiling：这讲真正的大招

课程最后把最大篇幅留给 tiling。因为在矩阵乘法里，真正支撑高性能的往往就是：**把大矩阵切成较小 tile，搬进 shared memory，重复利用，再分阶段累计结果**。这样做可以显著减少对 global memory 的重复读取，并让访问更容易 coalesce。

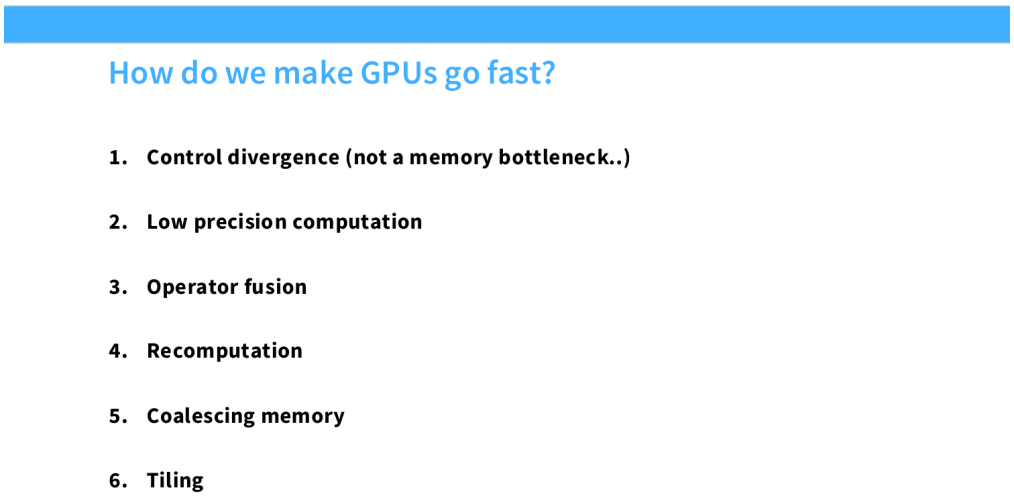

图 5: 课程把 GPU 提速方法浓缩成六条，其中真正牵一发动全身的是 tiling。

第 5 讲真正想让你带走的优化观

绝大多数 GPU 优化技巧，本质上都在做两件事之一：减少数据搬运，或者让数据搬运更符合硬件喜欢的方式。控制分支、低精度、fusion、recomputation、coalescing、tiling，本质上都可以放进这个统一框架。

4.7 本章小结

这些技巧看起来各不相同，但背后的共同语言其实就是 arithmetic intensity 与 memory hierarchy。只要你牢牢记住“compute 在涨，memory 更慢”，这些技巧为什么有效就会变得非常自然。

5 从矩阵乘法到 FlashAttention：GPU 直觉如何落到真实 LM kernel

5.1 为什么这讲最后要回到矩阵乘法之谜

课程后半段有一个很好的设计：它不是让学生记住一串优化技巧就结束，而是把这些技巧重新拉回一个具体问题，“为什么有时更大的矩阵反而更快?”这说明 GPU 性能不只是“规模大就慢，规模小就快”这种线性直觉，而是会受到 tiling、对齐、warp 调度和 wave quantization 等因素的共同影响。

5.2 FlashAttention 的意义：让 attention 更像硬件愿意接受的矩阵程序

虽然这讲只是预告性质地“unpack FlashAttention”，但它真正想传递的是：FlashAttention 的伟大之处，不只在于减少了某个 big-O，而是它把注意力计算重新组织成了更符合 GPU memory hierarchy 的形状。换句话说，它是一种典型的“respect the hardware”式算法设计。

为什么第 5 讲和第 6 讲是天然连续的

第 5 讲给的是硬件直觉和性能语言，第 6 讲才会进一步进入 kernel 层。没有这讲的 memory wall、tiling 和 coalescing 直觉，后面很多 kernel 技巧都会显得像魔法配方。

5.3 本章小结

第 5 讲最后的真正收束点，不是某个具体 GPU 参数，而是一个方法论：只有当你把算法写成硬件喜欢执行的样子，理论 FLOPs 才有可能变成真实吞吐。FlashAttention 只是这条原则最知名的例子之一。

6 总结与延伸

第 5 讲最重要的收获，可以概括成四个判断。

第一，**语言模型的发展越来越依赖 accelerator scaling**。没有 GPU / TPU 这类硬件的并行扩展，今天的大模型训练几乎不可能存在。

第二，**GPU 是吞吐机器，而不是低延迟机器**。这意味着你必须把问题写成大量相似、规则、可并行的计算，才能真正发挥它的优势。

第三，**memory wall 是现代 GPU 性能工程的中心矛盾**。不是算力不够，而是数据送不进去、搬不回来、重复搬太多。

第四，**几乎所有高性能 GPU 算法设计，都可以被理解为围绕 memory hierarchy 做出的取舍**。控制分支、低精度、fusion、recomputation、coalescing、tiling，全都如此。

我对第 5 讲的压缩提醒

如果你只把 GPU 当成“能跑 PyTorch 的设备”，那后面的 kernel、parallelism 和 inference 优化会一直像黑魔法；只有把它看成一台有明确执行与存储层次的机器，这些技巧才会真正变成可推理的工程知识。

从后续学习路径看，第 5 讲之后最自然的下一步就是第 6 讲的 kernel 与 Triton，因为你已经拿到了硬件直觉，现在需要把这种直觉真正转写成程序级优化。再往后，并行训练和推理系统则会继续把这个“硬件尊重”原则扩展到多卡和服务端场景。